

AWS CERTIFICATION EXAM PREP



AIF-C01

AWS Certified AI Practitioner

Complete Exam Notes

Domain-by-domain breakdown, exam-trigger tables, and a cheatsheet for every domain — condensed for exam week.

Notes by Mohammed Abdullah Khan · 2× AWS Certified

TABLE OF CONTENTS

AWS Certified AI Practitioner (AIF-C01) — Complete Notes

Domain 1: Fundamentals of AI and ML (20%)

- 1.1 The Hierarchy
- 1.2 Types of Machine Learning
- 1.3 Core ML Concepts
- 1.4 When AI/ML Is (and Isn't) the Right Tool
- 1.5 The ML Development Lifecycle (High Level)
- 1.6 AWS AI/ML Services Landscape (the "pre-built AI" layer)

 Domain 1 Cheatsheet

Domain 2: Fundamentals of Generative AI (24%)

- 2.1 Foundational Terms
- 2.2 Prompt Engineering
- 2.3 The Three Customization Strategies (and how they differ)
- 2.4 Inference Parameters (you WILL get scenario questions on these)
- 2.5 GenAI Advantages, Challenges & Use Cases

 Domain 2 Cheatsheet

Domain 3: Applications of Foundation Models (28% — most points live here)

- 3.1 The One Sentence That Resolves Most Questions
- 3.2 Amazon SageMaker — The ML Lifecycle Platform
- 3.3 Amazon Bedrock — Managed Foundation Models
- 3.4 SageMaker vs Bedrock — Side-by-Side
- 3.5 Amazon Q Family
- 3.6 Other AWS AI Services Relevant Here
- 3.7 Evaluation & Metrics (the part where nuance matters most)

 Domain 3 Cheatsheet

Domain 4: Guidelines for Responsible AI (14%)

- 4.1 Core Responsible AI Dimensions AWS Tests On
- 4.2 The Tool-to-Concern Mapping (memorize this table cold)
- 4.3 Interpretability vs Performance Trade-off
- 4.4 Environmental & Sustainability Considerations
- 4.5 Legal, IP, and Data Considerations

 Domain 4 Cheatsheet

Domain 5: Security, Compliance, and Governance for AI Solutions (14%)

- 5.1 The Standard Security Toolkit Applied to AI/ML

5.2 Logging vs Prevention (a frequently tested distinction)

5.3 Data Governance & Compliance

5.4 Shared Responsibility Model for AI Solutions

5.5 Governance for AI Specifically

 Domain 5 Cheatsheet

Master Quick-Fire Contrast Table (cross-domain last-minute review)

Traps That Cost Points at the High-Score Level

Self-Test — Explain These Out Loud (Feynman Check)

AWS Certified AI Practitioner (AIF-C01) — Complete Notes

Organized by the 5 official exam domains. Each domain ends with a **cheatsheet** — the compressed version to re-read the morning of the exam.

Domain	Weight
1. Fundamentals of AI and ML	20%
2. Fundamentals of Generative AI	24%
3. Applications of Foundation Models	28%
4. Guidelines for Responsible AI	14%
5. Security, Compliance, and Governance for AI Solutions	14%

Domain 1: Fundamentals of AI and ML (20%)

1.1 The Hierarchy

AI $\supset$ ML $\supset$ Deep Learning $\supset$ Generative AI

- **AI** — the umbrella. Machines performing tasks that normally need human intelligence (reasoning, perception, decision-making).
- **ML** — a subset of AI. Systems that *learn patterns from data* instead of being explicitly programmed with rules.
- **Deep Learning** — ML using neural networks with multiple layers. Good at unstructured data (images, text, audio) where hand-crafted features don't scale.
- **Generative AI (GenAI)** — a subset of deep learning that *creates new content* (text, images, code, audio) rather than just classifying or predicting a number/label.

Exam framing: if a question describes a system that *outputs a new artifact* (a paragraph, an image, a summary), that's GenAI. If it *outputs a label, category, or number* (fraud/not-fraud, price prediction), that's traditional ML — even if a neural net is under the hood.

1.2 Types of Machine Learning

Type	How it learns	Example use case	AWS trigger phrase
Supervised learning	Learns from labeled data (input $\rightarrow$ known correct output)	Fraud detection, price prediction, spam classification	"we have historical labeled examples"
Unsupervised learning	Finds structure in unlabeled data — clustering, dimensionality reduction, anomaly detection	Customer segmentation, anomaly detection	"no labels available", "group similar items"
Reinforcement learning (RL)	An agent learns by taking actions in an environment and getting rewards/penalties	Game-playing agents, robotics, RLHF (used to align LLMs to human preference)	"agent," "reward," "trial and error," "optimal policy"

Type	How it learns	Example use case	AWS trigger phrase
Semi-supervised learning	Small amount of labeled data + large amount of unlabeled data	When labeling is expensive but you have some labeled seed data	"limited labeled data"

Self-supervised learning — a training approach (not a separate "type" in the exam's simple taxonomy) where the model generates its own labels from the input itself. This is *how LLMs are pre-trained* — predicting the next token doesn't need human-provided labels, the text itself is the label. Know this because it explains why FMs can be trained on massive raw internet text.

1.3 Core ML Concepts

Labeled vs unlabeled data - **Labeled** — each example has a known correct answer (the "ground truth"). Required for supervised learning. - **Unlabeled** — raw data with no answer key. Used in unsupervised/self-supervised learning.

Structured vs unstructured data - **Structured** — rows/columns, fits a database table (transactions, sensor readings). - **Unstructured** — text, images, audio, video — no fixed schema. This is GenAI/deep learning's home turf.

Training vs Validation vs Test sets - **Training set** — what the model learns from. - **Validation set** — used *during* development to tune hyperparameters and pick the best model version. - **Test set** — held out, touched only at the end, gives an unbiased estimate of real-world performance. - Trigger: a question describing "data leakage" (test data accidentally seen during training) is testing whether you know these must stay separate.

Overfitting vs Underfitting (the bias-variance tradeoff)

Problem	What's happening	Symptom	Fix
Overfitting	Model memorizes training data, including noise	Great training accuracy, poor test/validation accuracy	More data, regularization, simplify model, early stopping, dropout
Underfitting	Model is too simple to capture the pattern	Poor accuracy on <i>both</i> training and test	More complex model, more/better features, train longer

Regularization — techniques (L1/L2 penalties, dropout) that discourage the model from over-relying on any single feature, reducing overfitting.

Bias vs Variance - **High bias** = underfitting (model's assumptions are too rigid). - **High variance** = overfitting (model is too sensitive to training data specifics). - Goal: minimize both — the classic tradeoff curve.

Inferencing — using a *trained* model to make predictions on new data. Two types: - **Real-time inference** — immediate response needed. - **Batch inference** — process a large volume of data with no urgency, no persistent endpoint. (These map directly to SageMaker’s real-time vs batch transform deployment options — see Domain 3.)

1.4 When AI/ML Is (and Isn’t) the Right Tool

- Use ML when: the pattern is too complex for hand-written rules, you have (or can get) representative data, and some error tolerance is acceptable.
- **Don’t** use ML when: a simple deterministic rule/if-else solves it, you need 100% explainable/auditable logic (e.g., regulatory compliance calculations), or you have no data.
- **Exam trap:** a question describing a simple, static, rule-based calculation (e.g., “apply a 10% discount if order > \$100”) is *not* an ML problem, even in an “AI” exam — the correct answer is often “this doesn’t need ML.”

1.5 The ML Development Lifecycle (High Level)

1. **Business problem framing** — define the problem and success metric *before* touching data.
2. **Data collection & preparation** — gather, clean, label, transform.
3. **Model selection & training** — pick an algorithm, train on prepared data.
4. **Evaluation** — validate against held-out data, business metrics.
5. **Deployment** — real-time endpoint, batch, edge, etc.
6. **Monitoring** — watch for drift, retrain as needed.

This lifecycle is the skeleton Domain 3 builds on — every SageMaker feature maps to exactly one of these stages.

1.6 AWS AI/ML Services Landscape (the “pre-built AI” layer)

These are **not** GenAI and **not** something you train yourself — fully managed, pre-trained AI services you call via API. Know them as a *contrast* to Bedrock/SageMaker.

Service	What it does	Exam trigger
Amazon Comprehend	NLP — sentiment analysis, entity recognition, key phrase extraction, PII detection	“analyze sentiment,” “extract entities from text” — not generative , an “understanding” service
Amazon Transcribe	Speech-to-text	“convert audio to text,” “call center transcription”
Amazon Polly	Text-to-speech	“convert text to natural-sounding speech”

Service	What it does	Exam trigger
Amazon Translate	Language translation	"translate content into multiple languages"
Amazon Rekognition	Computer vision — image/video analysis, object/face detection, content moderation	"detect objects in images," "moderate uploaded images"
Amazon Textract	Extracts text/data from scanned documents and forms	"extract data from PDFs/scanned forms"
Amazon Lex	Build conversational chatbots/voice bots (same tech as Alexa)	"build a chatbot" (older/non-FM approach)
Amazon Personalize	Real-time personalized recommendations	"product recommendations," "personalization engine"
Amazon Forecast	Time-series forecasting	"forecast demand/sales"
Amazon Kendra	Intelligent enterprise search	"enterprise search," often a retriever component in RAG alongside Bedrock

Exam pattern: these older, narrow "AI Services" are frequently posed against Bedrock in "which service would you use" questions. If the need is narrowly one task (sentiment, translation, transcription) and doesn't require generating novel content or a conversational LLM, the pre-built AI service is usually *cheaper and simpler* than standing up a Bedrock/GenAI solution — pick the narrow tool.

CHEAT SHEET

Domain 1 Cheatsheet

- **AI** ⊃ **ML** ⊃ **Deep Learning** ⊃ **GenAI** — nesting, not synonyms.
- Outputs a *new artifact* → GenAI. Outputs a *label/number* → traditional ML.
- **Supervised** = labeled data → predict. **Unsupervised** = unlabeled → find structure. **Reinforcement** = agent + reward/penalty. **Self-supervised** = model makes its own labels (how LLMs pre-train).
- **Overfitting** = memorized noise, great on train/bad on test → fix with more data/regularization/simpler model.
- **Underfitting** = too simple, bad on both → fix with more complexity/features.
- High bias = underfitting. High variance = overfitting.
- Train / Validation / Test sets must stay separate — mixing them = data leakage.

- Real-time inference = immediate answer needed. Batch inference = large volume, no urgency.
- If a scenario is a simple deterministic rule, the right answer might be "you don't need ML."
- Comprehend = NLP understanding (not generative). Rekognition = vision. Transcribe = speech→text. Polly = text→speech. Translate = language. Textract = documents. Personalize = recs. Forecast = time-series. Lex = chatbots (non-FM). Kendra = enterprise search (RAG retriever).
- ML lifecycle: **Frame problem → Prepare data → Train → Evaluate → Deploy → Monitor.**

Domain 2: Fundamentals of Generative AI (24%)

2.1 Foundational Terms

Foundation Model (FM) — a large model pre-trained on massive, broad datasets that can be adapted (via prompting, fine-tuning, or RAG) to many downstream tasks. Key exam idea: FMs are *general-purpose*; you customize them rather than training from scratch.

LLM (Large Language Model) — a foundation model specialized in text. Trained on huge text corpora to predict the next token given previous tokens (**autoregressive**).

Token — the basic unit a model processes, roughly a word-piece (not always a full word). Pricing, context windows, and latency are usually measured in tokens.

Context window — the max number of tokens (input + output combined, in most architectures) a model can “see” at once. Bigger context window = can ingest longer documents/conversations, but cost and latency scale with it.

Embeddings — numerical vector representations of text/images that capture semantic meaning — similar meanings → vectors close together in vector space. Core to search, recommendation, and RAG.

Vector Database — a database optimized for storing embeddings and doing similarity search (e.g., cosine similarity, k-NN). AWS options: **Amazon OpenSearch Service** (k-NN plugin), **Amazon Aurora + pgvector**, **Amazon RDS for PostgreSQL + pgvector**, **Amazon Neptune Analytics**, **MemoryDB for Redis** (vector search), **Amazon Kendra** (not a vector DB itself, but relevant for retrieval).

Multimodal models — models that accept/produce more than one modality (text + image, text + audio). Amazon Nova, Claude, and Titan Multimodal Embeddings are examples referenced in Bedrock.

Hallucination — when a model generates plausible-sounding but factually incorrect or fabricated content. Mitigations: RAG, lower temperature, human-in-the-loop review, guardrails.

Diffusion models — generate images by starting from random noise and iteratively denoising it into a coherent image (used by Stable Diffusion / Titan Image Generator).

GANs (Generative Adversarial Networks) — two networks (generator + discriminator) compete — generator tries to fool the discriminator into thinking generated data is real. Older image-gen approach, still tested conceptually.

Transformer architecture — the architecture behind modern LLMs. Key concept: **self-attention** — lets the model weigh the importance of every other token in the sequence when processing a given token, regardless of distance. This is *why* transformers handle long-range context better than older RNN/LSTM approaches.

2.2 Prompt Engineering

The input given to a model, and the discipline of crafting inputs to get better outputs **without changing model weights**. Techniques you should know cold:

- **Zero-shot** — no examples given, just the instruction.
- **Few-shot** — a handful of examples included in the prompt to steer format/behavior.
- **Chain-of-thought (CoT)** — asking the model to reason step-by-step before answering (improves multi-step reasoning).
- **Prompt templates** — reusable structured prompts with variable slots.
- **Negative prompting** — telling the model what *not* to do/include (more common in image gen).

Prompt injection — a security concern (see Domain 5) where malicious input tries to override the system prompt/instructions. Mitigated via Guardrails and input sanitization.

2.3 The Three Customization Strategies (and how they differ)

Approach	Changes model weights?	Solves	Cost/complexity	Data needed
Prompt engineering	No	Quick behavior tweak, no training data available	Cheapest, fastest	None
RAG (Retrieval-Augmented Generation)	No	Model needs facts that change often / company-private docs	Medium — needs a vector store + retrieval pipeline	Your document corpus (not labeled)
Fine-tuning	Yes	Model needs to learn a new <i>style</i> , <i>format</i> , or <i>domain jargon</i> permanently	Higher — needs labeled data + training compute	Labeled input/output pairs

RAG (Retrieval-Augmented Generation) — instead of relying purely on the model's frozen training data, you retrieve relevant documents (via vector search) at query time and inject them into the prompt as context. Solves hallucination and stale-knowledge problems *without* retraining. AWS service: **Amazon Bedrock Knowledge Bases**.

Fine-tuning — continuing to train a pre-trained FM on your own labeled dataset to specialize it. Types to distinguish: - **Full fine-tuning** — updates all model weights (expensive, needs lots of data). - **Instruction fine-tuning** — trains on instruction/response pairs to make the model better at following directions. - **PEFT (Parameter-Efficient Fine-Tuning)**, e.g., **LoRA** — freezes most weights, trains a small set of new parameters. Much cheaper. - **Continued pre-training** (Bedrock term) — feeding

more domain-specific *unlabeled* data before fine-tuning, to adapt the base knowledge (e.g., legal, medical domain adaptation).

The exam's favorite decision tree: - Need the model to *know facts* it doesn't have / stay current →

RAG - Need the model to *behave/respond differently* (tone, format, specialized task) → **Fine-tuning** -

Need a *quick* behavior tweak with no training data → **Prompt engineering** - "Our data changes frequently" → **RAG** (fine-tuning would require constant retraining) - "The model needs to adopt a specific tone/format permanently" → **Fine-tuning**

2.4 Inference Parameters (you WILL get scenario questions on these)

- **Temperature** — randomness/creativity control. 0 = deterministic/focused, higher = more diverse/creative.
- **Top-p (nucleus sampling)** — model considers only the smallest set of tokens whose cumulative probability $\geq p$ (dynamic pool size).
- **Top-k** — model considers only the k most likely next tokens (fixed number of candidates).
- **Max tokens / length** — caps output length.

2.5 GenAI Advantages, Challenges & Use Cases

Advantages: adaptability across tasks, personalization at scale, automation of content creation, faster time-to-value than building custom models, can operate across data types (multimodal).

Challenges/limitations: hallucination, non-determinism (same prompt $\neq$ same output every time unless temperature=0), cost/latency at scale, regulatory/IP uncertainty around training data and generated content, difficulty fully explaining *why* an output was produced (black box).

Common GenAI use cases (exam loves matching these to a technique): - Content generation/summarization → prompt engineering / base FM. - Customer service chatbots grounded in company docs → RAG. - Code generation → FM with coding specialization (e.g., Amazon Q Developer). - Search & recommendations → embeddings + vector search. - Domain-specific tone/format (e.g., legal document drafting in firm's style) → fine-tuning.

CHEAT SHEET

Domain 2 Cheatsheet

- **FM** = general-purpose pre-trained model. **LLM** = FM specialized in text, predicts next token (autoregressive).
- **Token** $\approx$ word-piece. **Context window** = max tokens model sees at once (input+output).
- **Embeddings** = vectors capturing meaning; similar meaning → close vectors. Stored in a **vector database** (OpenSearch, Aurora/RDS+pgvector, Neptune Analytics, MemoryDB, Kendra as retriever).
- **Self-attention** = why transformers beat RNN/LSTM at long-range context.
- **Hallucination** fix priority: RAG > lower temperature > human review > guardrails.

- **Diffusion models** = noise → image (Stable Diffusion, Titan Image Gen). **GANs** = generator vs discriminator (older).
- Prompting techniques: **zero-shot, few-shot, CoT, templates, negative prompting** — none change model weights.
- **RAG** changes *what the model knows* (facts) without touching weights. **Fine-tuning** changes *how it behaves* (style/format) by updating weights.
- Fine-tuning data changes often → prefer RAG instead (avoid constant retraining).
- **LoRA/PEFT** = cheap fine-tuning, freezes most weights.
- **Continued pre-training** = unlabeled domain data, adapts base knowledge before fine-tuning.
- **Temperature** 0=deterministic, high=creative. **Top-p** = cumulative probability threshold (dynamic). **Top-k** = fixed candidate count.
- Decision tree: facts/current data → RAG. Style/behavior → fine-tune. Quick tweak, no data → prompt engineering.

Domain 3: Applications of Foundation Models (28% — most points live here)

3.1 The One Sentence That Resolves Most Questions

Bedrock = you want a foundation model, fast, managed, no infra. SageMaker = you want to build/train/deploy your own model (or heavily customize one), full control over the ML lifecycle.

AWS exam-writers reward the *more managed* option when both look plausible. If a question is genuinely “give me access to Claude/Titan/Llama and let me build a GenAI app,” the answer is Bedrock even if SageMaker *could* technically do it. If the question involves your own labeled data, custom algorithms, traditional ML (regression, classification, forecasting), or full training-pipeline control, it’s SageMaker.

3.2 Amazon SageMaker — The ML Lifecycle Platform

Think of SageMaker as “AWS’s answer to every stage of the ML pipeline: *prepare* → *build* → *train* → *deploy* → *monitor*.” Each component maps to one stage. Memorize by stage, not by name — that’s how the exam scenario-questions are actually built.

PREPARE / LABEL DATA

Tool	What it does	Exam trigger phrase
SageMaker Data Wrangler	Visual data prep, cleaning, feature transforms, bias detection during prep	“clean/transform data before training”
SageMaker Ground Truth	Human-in-the-loop data labeling , uses active learning to reduce human effort over time	“label training data”, “need humans to annotate”
SageMaker Feature Store	Centralized repo for storing/sharing/reusing ML features across teams, online (low-latency) + offline (batch/training) store	“reuse features across models”, “avoid feature drift between training/inference”

BUILD / TRAIN

Tool	What it does	Exam trigger phrase
SageMaker Studio	The IDE — unified web UI for the whole lifecycle	"single interface for ML workflow"
SageMaker Autopilot	AutoML — automatically builds, trains, tunes models, gives you a leaderboard + notebook it generated	"no ML expertise", "automatically explore algorithms"
SageMaker JumpStart	Pre-built/pre-trained models + solution templates you can fine-tune or deploy directly (this is SageMaker's overlap zone with Bedrock — JumpStart hosts open FMs like Llama, Mistral, Stable Diffusion, whereas Bedrock hosts a curated set behind a single managed API)	"quickly deploy a pre-trained open-source model", "one-click deploy"
Built-in algorithms	XGBoost, Linear Learner, etc. — no need to write training code	"traditional ML with tabular data"
SageMaker Experiments	Track/compare training runs, hyperparameters, metrics	"compare multiple training runs"
SageMaker Debugger	Real-time monitoring <i>during training</i> for issues like vanishing gradients, overfitting	"training job is misbehaving mid-run"
Hyperparameter tuning (HPO)	Automatic search (Bayesian, random, grid) for best hyperparameters	"optimize hyperparameters automatically"
Distributed training	Data parallel / model parallel libraries for large models across multiple GPUs/instances	"training won't fit on one GPU"

DEPLOY

Tool	What it does	Exam trigger phrase
Real-time endpoints	Persistent, low-latency inference, auto-scaling	"consistent traffic", "millisecond latency"

Tool	What it does	Exam trigger phrase
Serverless inference	No infra management, scales to zero, cold-start tradeoff	"intermittent/unpredictable traffic", "cost efficiency, don't want idle capacity"
Batch transform	Offline inference on large datasets, no persistent endpoint	"process a large dataset all at once, no need for real-time"
Asynchronous inference	Queue-based, for large payloads or long processing times, auto-scales to zero	"large payload (video, big documents)", "long inference time, don't need response instantly"
Multi-model endpoints	Host multiple models on one endpoint to save cost	"many models, low individual traffic each"
SageMaker Neo	Compile models to run efficiently on specific hardware (edge devices, specific instance types)	"optimize model for edge deployment"

MONITOR / GOVERN

Tool	What it does	Exam trigger phrase
SageMaker Model Monitor	Detects drift in production — data quality drift, model quality drift, bias drift, feature attribution drift, <i>after</i> deployment	"model degraded over time in production"
SageMaker Clarify	Detects bias (pre-training AND post-training) and provides explainability (feature importance via SHAP)	"detect bias in dataset/model", "explain why model made a prediction"
SageMaker Model Registry	Version control + approval workflow for models before deployment	"manage model versions", "approval before production"
SageMaker Pipelines	CI/CD / orchestration for ML workflows (the MLOps piece)	"automate the ML workflow end-to-end"
SageMaker Model Cards	Documentation of a model's intended use, limitations, training data, risk considerations	"document model for governance/audit"

Clarify vs Model Monitor — the #1 confused pair: - **Clarify** = bias & explainability, can run **before** training (on data) or **after** (on model) — it's about *fairness and "why."* - **Model Monitor** = ongoing production drift detection — it's about *"has anything changed since deployment?"*

3.3 Amazon Bedrock — Managed Foundation Models

Bedrock's entire value proposition: **access multiple FM providers through one API, with no infrastructure to manage, and enterprise controls (security, guardrails, customization) wrapped around it.**

CORE BUILDING BLOCKS

Model access - Multiple providers in one place: Anthropic (Claude), Amazon (Titan/Nova), Meta (Llama), Mistral, Cohere, AI21, Stability AI. - **On-Demand** pricing = pay per token/request, no commitment. - **Provisioned Throughput** = reserve capacity for guaranteed throughput (needed for high, consistent volume, or for custom/fine-tuned models — most fine-tuned models *require* Provisioned Throughput to serve). - **Batch inference** = submit large jobs asynchronously at lower cost, non-real-time.

APIs - `InvokeModel` — single request/response. - `Converse` / `ConverseStream` — the newer unified API AWS wants you to use across all model providers, standardizes the conversation format (this is the current recommended API — good to know for exam recency).

Knowledge Bases for Amazon Bedrock (RAG) - Fully managed **RAG**: connects to a vector store (OpenSearch Serverless, Aurora, Pinecone, Redis, MongoDB), handles chunking + embedding + retrieval automatically. - Trigger phrase: "ground responses in company data without retraining the model," "reduce hallucinations using our documents."

Agents for Amazon Bedrock - Lets an FM **take multi-step actions** — call APIs/Lambda functions ("action groups"), orchestrate tasks, maintain session state. - Trigger phrase: "the model needs to take actions," "call internal systems/APIs," "multi-step task automation."

Guardrails for Amazon Bedrock - Content filtering: denied topics, PII redaction, profanity/harmful content filters, word filters, contextual grounding checks (catches hallucinations against your source data), configurable per use case. - Trigger phrase: "prevent the model from discussing X," "filter harmful/toxic output," "redact PII from responses." - **This is the default answer whenever a question asks "how do we control what the model says."**

Model customization (in order of how much they change the model) 1. **Prompt engineering** — cheapest, fastest, no data needed. 2. **RAG (Knowledge Bases)** — grounds the model in your data, doesn't change model weights. 3. **Fine-tuning** — labeled data, updates model weights for style/format/behavior. 4. **Continued pre-training** — unlabeled domain-specific data, adapts the model to a whole domain before fine-tuning.

Model Evaluation - Bedrock has built-in **model evaluation** jobs: automatic (using benchmark metrics) or human-based (your own team reviews outputs) — useful for comparing models before picking one.

Custom Model Import — bring your own already fine-tuned model into Bedrock to serve it via the managed API.

Watermarking / Content provenance - Amazon Titan Image Generator embeds invisible watermarks by default — relevant for “how do we detect AI-generated content” questions.

3.4 SageMaker vs Bedrock — Side-by-Side

Dimension	SageMaker	Bedrock
Best for	Custom ML models, traditional ML, full control	Foundation models, GenAI apps, fast time-to-value
Infra management	You manage instances/endpoints (unless serverless)	Fully serverless, zero infra
Model source	Train your own, or JumpStart pre-trained models	Curated FMs from AWS + partners
Customization	Full (architecture, training code, hyperparameters)	Fine-tuning / continued pre-training within provider limits
Typical exam cue	“labeled dataset,” “custom algorithm,” “traditional ML,” “full lifecycle”	“chatbot,” “generate text/images,” “RAG,” “agents,” “guardrails”

JumpStart is the trap: it lives *inside* SageMaker but deploys pre-trained FMs — so a question can describe FM deployment and still have SageMaker JumpStart as the correct answer if the scenario emphasizes needing infrastructure control, specific instance types, or a model not available in Bedrock’s provider list.

3.5 Amazon Q Family

- **Amazon Q Business** — enterprise assistant that answers questions grounded in your company's internal data/documents.
- **Amazon Q Developer** (formerly CodeWhisperer) — coding assistant: code suggestions, security scanning, code transformation (e.g., Java version upgrades).
- Neither is SageMaker or Bedrock directly, but they’re **built on top of Bedrock** — the exam sometimes tests this relationship.

3.6 Other AWS AI Services Relevant Here

- **Amazon Kendra** — intelligent enterprise search, often used as a retriever component in RAG architectures alongside Bedrock.
- **AWS Trainium / Inferentia** — custom silicon chips: Trainium for training, Inferentia for inference — cost/performance optimization angle, likely to appear as “how do you reduce training/inference cost” answer options.

3.7 Evaluation & Metrics (the part where nuance matters most)

A. AUTOMATED TEXT-GENERATION METRICS

Compare generated text to a reference/"gold" text. Know what each measures and its blind spot.

Metric	What it measures	Typical use case	Weakness
BLEU	N-gram precision overlap between generated and reference text	Machine translation	Penalizes valid paraphrasing; no semantic understanding
ROUGE	N-gram/sequence <i>recall</i> overlap (ROUGE-N, ROUGE-L)	Summarization	Same surface-overlap limitation as BLEU
METEOR	Like BLEU but accounts for synonyms/stemming	Translation	Still surface-level, better than BLEU for synonym cases
Perplexity	How "surprised" a model is by the next token — lower = model predicts the sequence more confidently	Comparing language models' fluency	Doesn't measure factual correctness or usefulness
BERTScore	Uses embeddings to compare semantic similarity of generated vs reference text	Cases where wording differs but meaning should match	Needs a reference embedding model, more compute

B. CLASSIFICATION-STYLE METRICS (RELEVANT WHEN GENAI OUTPUT IS EVALUATED AS RIGHT/WRONG, OR FOR THE UNDERLYING CLASSIFIERS)

- **Accuracy** — correct predictions / total predictions. Misleading on imbalanced data.
- **Precision** — of what the model flagged positive, how much was actually positive. (Minimizes false positives.)
- **Recall (Sensitivity)** — of all actual positives, how much did the model catch. (Minimizes false negatives.)
- **F1 Score** — harmonic mean of precision and recall — use when you need a single number balancing both.
- **Confusion Matrix** — table of TP/FP/TN/FN, the source for all the above.
- **AUC-ROC** — how well a model separates classes across all thresholds — closer to 1 is better.

Exam framing tip: if a question emphasizes "cost of missing a positive case is high" (e.g., fraud, cancer detection) → **recall** matters most. If "cost of a false alarm is high" (e.g., spam folder losing real emails) → **precision** matters most.

C. BUSINESS/HUMAN-CENTERED EVALUATION

- **Human evaluation** — people rate outputs on helpfulness, coherence, correctness, tone. Gold standard but slow/expensive; Bedrock's Model Evaluation supports this as a job type alongside automatic metrics.
- **RAGAS-style RAG metrics** (conceptually testable even if not named): **faithfulness** (does the answer stick to retrieved context, i.e., not hallucinate), **context relevance** (did retrieval fetch the right documents), **answer relevance** (does the answer address the actual question).
- **A/B testing** — comparing model/prompt versions with live traffic against a business KPI (conversion, engagement, task completion).
- **Latency, throughput, cost per inference** — non-quality but exam-relevant "performance" metrics, especially in Bedrock Provisioned Throughput vs on-demand questions.

D. BUSINESS METRICS TIED TO ROI

- **Cross-domain performance** — how well a single model performs across different task types.
- **Efficiency** — cost and time to reach a target performance level.
- **Conversion rate, average revenue per user, customer lifetime value** — the kind of KPIs that show up when a question asks "how do you measure the business impact of a GenAI deployment," as opposed to model-quality metrics.

CHEAT SHEET

Domain 3 Cheatsheet

- **Bedrock** = pre-trained FM via managed API, minimal ML expertise needed. **SageMaker** = full build/train/deploy custom ML pipeline.
- SageMaker by stage: **Prepare** (Data Wrangler, Ground Truth, Feature Store) → **Build/Train** (Studio, Autopilot, JumpStart, Experiments, Debugger, HPO) → **Deploy** (real-time, serverless, batch transform, async, multi-model, Neo) → **Monitor/Govern** (Model Monitor, Clarify, Model Registry, Pipelines, Model Cards).
- **Clarify** = bias + explainability (before/after training). **Model Monitor** = production drift over time.
- Bedrock features: **Knowledge Bases** = managed RAG. **Agents** = multi-step actions/API calls. **Guardrails** = runtime content filtering. **Model Evaluation** = offline benchmarking (auto or human). **Provisioned Throughput** = reserved capacity, required for most fine-tuned models. **Custom Model Import** = bring your own fine-tuned model.
- **JumpStart is the trap** — FM deployment question can still have SageMaker JumpStart as the answer if infra control / specific instance / non-Bedrock model is emphasized.
- Inference deployment pattern-match: steady traffic = **real-time endpoint**; unpredictable/spiky = **serverless**; huge payload/long-running = **asynchronous**; no urgency/big dataset = **batch transform**; many low-traffic models = **multi-model endpoint**; edge/hardware-specific = **Neo**.

- **Amazon Q Business** = enterprise data assistant. **Amazon Q Developer** = coding assistant. Both built on Bedrock.
- **Kendra** = enterprise search, common RAG retriever. **Trainium** = training chip. **Inferentia** = inference chip.
- Text-gen metrics: **BLEU/ROUGE/METEOR** = surface n-gram overlap (blind to paraphrasing). **Perplexity** = fluency/confidence only. **BERTScore** = semantic similarity via embeddings.
- Classification: **Precision** = trustworthy positives (minimize false alarms). **Recall** = catch all positives (minimize misses). **F1** = balance of both. High cost of missing a positive → recall. High cost of false alarm → precision.
- RAG-specific quality: **faithfulness** (no hallucination vs context), **context relevance** (right docs retrieved), **answer relevance** (answers the actual question).
- Business impact ≠ model quality: conversion rate, ARPU, CLV, A/B testing outcomes are the "ROI" answer when a question isn't asking about accuracy.

Domain 4: Guidelines for Responsible AI (14%)

4.1 Core Responsible AI Dimensions AWS Tests On

- **Fairness / Bias** — model performs unevenly across demographic groups. Mitigate via diverse training data, **SageMaker Clarify**.
- **Explainability / Transparency** — ability to understand *why* a model produced an output. **SageMaker Clarify** again (SHAP values), plus “model cards.”
- **Robustness** — model performs reliably under varied/adversarial inputs.
- **Privacy & security** — PII protection, data encryption, access control (IAM), **Bedrock Guardrails** for PII redaction.
- **Safety** — preventing harmful, dangerous, or toxic outputs.
- **Governance** — documentation, audit trails, approval workflows for model deployment. **Amazon SageMaker Model Cards** and **Model Registry** are the AWS artifacts for this.

AWS’s own framework reference: AWS Responsible AI dimensions (Fairness, Explainability, Privacy & Security, Robustness, Safety, Governance, Transparency, Veracity/Accuracy, Controllability) — a question might list these and ask you to categorize a scenario correctly.

4.2 The Tool-to-Concern Mapping (memorize this table cold)

Concern	AWS answer
Bias (data or model)	SageMaker Clarify
Explainability (“why did it predict this”)	SageMaker Clarify (SHAP values)
Harmful/toxic content generation	Bedrock Guardrails
PII leakage in output	Bedrock Guardrails (PII redaction)
Human review of low-confidence predictions	Amazon A2I (Augmented AI)
Model card / documentation of intended use, limitations	SageMaker Model Cards
Production drift after deployment	SageMaker Model Monitor
Version control / approval before production	SageMaker Model Registry

A2I vs Clarify vs Guardrails — don’t let “responsible AI” collapse these into one answer: - **A2I** = human review loop (routes uncertain predictions to a person). - **Clarify** = bias/explainability (pre- or

post-training analysis). - **Guardrails** = real-time content filtering at inference.

4.3 Interpretability vs Performance Trade-off

- **Transparent/"white box" models** (linear regression, decision trees) — easy to explain, but often lower accuracy on complex problems.
- **Deep learning / "black box" models** — higher performance on complex/unstructured data, but much harder to interpret.
- Expect a question contrasting this trade-off directly: choosing a simpler, more explainable model *deliberately sacrifices* some accuracy when regulatory/audit explainability is the priority.

4.4 Environmental & Sustainability Considerations

- Training large FMs consumes significant compute/energy. AWS's answer angle: use **smaller/right-sized models when possible**, **Trainium/Inferentia** for efficiency, and **SageMaker's managed spot training** to reduce compute waste.
- If a question frames "reduce environmental impact of ML training," the answer favors efficient hardware (Trainium/Inferentia), right-sizing the model, and avoiding unnecessary retraining — not "train a bigger model for better accuracy."

4.5 Legal, IP, and Data Considerations

- **Training data provenance** — using copyrighted/licensed data to train a model without rights can create legal exposure; a responsible-AI-minded answer favors licensed/owned/properly consented data.
- **Output ownership/IP** — generated content's IP status can be ambiguous; guardrails and human review reduce risk of publishing infringing or non-compliant content.
- **Bias amplification** — models trained on biased historical data will reproduce/amplify that bias unless actively mitigated (Clarify, diverse data, fairness constraints).

CHEAT SHEET

Domain 4 Cheatsheet

- Core dimensions: **Fairness, Explainability, Robustness, Privacy/Security, Safety, Governance, Transparency.**
- **Bias/Explainability → Clarify. Toxic content/PII in output → Guardrails. Human review of uncertain predictions → A2I. Documentation/versioning → Model Cards + Model Registry. Post-deployment drift → Model Monitor.**
- A2I = human-in-the-loop. Clarify = bias & "why." Guardrails = runtime filtering. Keep these three distinct — the exam actively tries to blur them.
- Interpretable models (linear/decision tree) trade accuracy for explainability; deep learning trades explainability for accuracy — pick based on whether the scenario needs audit-grade transparency.

- Environmental impact question → favor **Trainium/Inferentia**, right-sized models, spot training — not “bigger model.”
- Training data legal exposure = use licensed/owned data; bias in training data = actively mitigate via Clarify + diverse data, don’t assume it self-corrects.

Domain 5: Security, Compliance, and Governance for AI Solutions (14%)

5.1 The Standard Security Toolkit Applied to AI/ML

- **IAM** — roles/policies scoped to specific SageMaker/Bedrock actions; **least privilege** is always the safe default answer.
- **KMS** — encryption at rest for training data, model artifacts, Bedrock model invocation logs.
- **VPC endpoints / PrivateLink** — keep traffic to SageMaker/Bedrock off the public internet.
- **CloudTrail** — audit logging of API calls (who invoked what model, when).
- **CloudWatch** — monitoring/metrics/logs for both services.
- **Bedrock model invocation logging** — can log full prompts/responses to S3/CloudWatch for audit — **distinct from Guardrails** (logging = record what happened, Guardrails = prevent it from happening).

Quick pattern: a generic “security question” → **IAM + KMS + CloudTrail + VPC endpoints** covers the large majority of correct answers.

5.2 Logging vs Prevention (a frequently tested distinction)

Need	AWS answer
Prevent harmful/off-topic/PII output in real time	Bedrock Guardrails
Record what was asked/answered for audit later	Bedrock model invocation logging (to S3/CloudWatch)
Record who called which API and when	CloudTrail

5.3 Data Governance & Compliance

- **Data residency/sovereignty** — choosing AWS Regions to keep data within required jurisdictions.
- **Encryption in transit and at rest** — TLS for transit, KMS-backed encryption for at rest, applies to training data, model artifacts, and logs.
- **Compliance standards** — AWS services generally inherit AWS’s compliance certifications (HIPAA, GDPR-relevant controls, SOC, PCI-DSS, FedRAMP) but the *customer* is responsible for configuring services compliantly — this is the **shared responsibility model** applied to AI.
- **AWS Artifact** — self-service portal for downloading AWS compliance reports/certifications — the answer when a question asks “where do I get proof AWS meets compliance standard X.”

5.4 Shared Responsibility Model for AI Solutions

- **AWS is responsible for:** security *of* the cloud — physical infrastructure, host OS patching for managed services, global network security.
- **Customer is responsible for:** security *in* the cloud — IAM policies, data classification/encryption choices, network configuration (VPCs/security groups), Guardrails/content policy configuration, model access permissions.
- **Exam trap:** don't pick "AWS handles it automatically" for things like IAM policy design, data encryption key management choices, or Guardrails configuration — those remain the customer's job even on fully-managed services like Bedrock.

5.5 Governance for AI Specifically

- **Approval workflows before production** — **SageMaker Model Registry** (also a Domain 3/4 tool, but the *governance* angle here is: nothing reaches production without sign-off).
- **Auditability of model decisions** — combination of CloudTrail (who/when), Bedrock invocation logging (what was asked/answered), and Model Cards (what the model is/isn't intended for).
- **Access control granularity** — IAM policies can scope down to specific model IDs in Bedrock (e.g., allow `InvokeModel` only for a specific Claude model ARN), not just service-level access.

CHEAT SHEET

Domain 5 Cheatsheet

- Default security stack for AI: **IAM (least privilege) + KMS (encryption) + CloudTrail (API audit) + VPC endpoints/PrivateLink (private traffic) + CloudWatch (monitoring).**
- **Guardrails = prevent** (runtime). **Invocation logging = record** (audit trail, sent to S3/CloudWatch). **CloudTrail = record who called what API.** Three different jobs — don't conflate.
- **Shared responsibility:** AWS secures *of* the cloud (infra); customer secures *in* the cloud (IAM, encryption config, Guardrails config, network setup) — even on Bedrock's serverless model.
- **AWS Artifact** = where to pull AWS's own compliance certifications/reports.
- Governance chain for auditability: **CloudTrail (who/when) + Invocation logging (what) + Model Cards (intended use) + Model Registry (approval gate).**
- IAM in Bedrock can scope to a **specific model ARN**, not just "Bedrock access" broadly — expect least-privilege questions to test this granularity.

Master Quick-Fire Contrast Table (cross-domain last-minute review)

Confusable Pair	Distinction
Fine-tuning vs RAG	Fine-tuning changes weights (style/behavior); RAG changes what info is available (facts) without touching weights
Bedrock vs SageMaker	Bedrock = access pre-trained FMs via API, minimal ML expertise needed; SageMaker = full build/train/deploy custom ML pipeline
Precision vs Recall	Precision = trustworthy positives; Recall = catching all positives
BLEU/ROUGE vs BERTScore	Surface n-gram overlap vs semantic embedding similarity
Top-p vs Top-k	Top-p = cumulative probability threshold (dynamic pool size); Top-k = fixed number of candidates
Amazon Q Business vs Q Developer	Business = enterprise data assistant; Developer = coding assistant
Guardrails vs Model Evaluation (Bedrock)	Guardrails = runtime content filtering; Model Evaluation = offline comparison/benchmarking tool
Guardrails vs Invocation Logging	Guardrails = prevent bad output; Invocation logging = record what happened, for audit
Clarify vs Model Monitor	Clarify = bias/explainability (before or after training); Model Monitor = ongoing drift <i>after</i> deployment
Clarify vs A2I vs Guardrails	Clarify = bias/"why"; A2I = human review loop; Guardrails = runtime filtering
Supervised vs Unsupervised vs RL	Labeled→predict; unlabeled→find structure; agent+reward→learn policy
Overfitting vs Underfitting	Overfit = memorized noise (good train/bad test); Underfit = too simple (bad on both)

Confusable Pair	Distinction
Real-time vs Serverless vs Async vs Batch inference	Steady traffic / unpredictable-spiky / huge-payload-long-running / no-urgency-bulk

Traps That Cost Points at the High-Score Level

1. **"Which is cheaper/faster to get started"** almost always → Bedrock (no infra, no training).
2. **Real-time vs Serverless vs Async vs Batch inference** — read for *traffic pattern* keywords.
3. **RAG vs Fine-tuning** — "data changes frequently" → RAG; "needs a specific tone/format permanently" → fine-tuning.
4. **A2I vs Clarify vs Guardrails** — don't let "responsible AI" as a category collapse these into one answer.
5. **Provisioned Throughput** — custom/fine-tuned Bedrock models generally *require* it; common "which pricing model" gotcha.
6. **Amazon Q Business vs Amazon Q Developer** — built on Bedrock, but distinct audiences (enterprise data vs coding).
7. **JumpStart vs Bedrock** — JumpStart wins when infra control, specific instance types, or a non-Bedrock model is emphasized.
8. **"AWS handles it automatically"** — wrong for IAM design, encryption key choices, Guardrails config, even on serverless Bedrock (shared responsibility).
9. A scenario describing a simple deterministic rule → correct answer can be "you don't need ML at all."

Self-Test — Explain These Out Loud (Feynman Check)

1. Why would you pick SageMaker JumpStart over Bedrock for a Llama model?
2. What's the difference between what Clarify does before training vs after training?
3. Why does Provisioned Throughput matter for fine-tuned Bedrock models specifically?
4. Give a scenario where RAG is wrong and fine-tuning is right.
5. What's the difference between Guardrails logging and Bedrock invocation logging?
6. Why is self-supervised learning the reason LLMs can pre-train on raw internet text?
7. Give an example where precision matters more than recall, and one where recall matters more.
8. Under the shared responsibility model, name three things that stay the customer's job even when using Bedrock.
9. What's the difference between A2I, Clarify, and Guardrails in one sentence each?
10. Why might "train a bigger model" be the *wrong* answer to a sustainability-focused question?

About the Author

Mohammed Abdullah Khan

AWS Certified AI Practitioner

AWS Certified Cloud Practitioner

I write concise, exam-focused AWS certification notes — distilled down to what's actually tested, organized by domain, with the exam traps called out explicitly. If these notes helped you pass, I'd love to hear about it.



Portfolio mohammedabdullahkhan.com



LinkedIn linkedin.com/in/mohammed-abdullah-khan-7b82a31a5



Email mohammedabdullahkhan26523@gmail.com

Good luck on your exam.